Git Cheatsheet

The Ultimate Reference

80+ Essential Commands

`Commands for Git v2.54.0`

By Inayatullah Shinwari

https://inayatullahshinwari.github.io/git-cheatsheet

Basics — Initialize, clone, and inspect

Initialize Repository
```
$ git init
```
Create a new Git repository in the current directory.

Clone Repository
```
$ git clone
```
Clone a remote repository to your local machine.

Clone Specific Branch
```
$ git clone -b
```
Clone a specific branch from a remote repository.

Check Status
```
$ git status
```
Show the working tree status — modified, staged, and untracked files.

View Log
```
$ git log
```
Show commit history with author, date, and message.

Pretty Log (One Line)
```
$ git log --oneline --graph --decorate
```
Compact, visual commit history with branch decorations.

Show Changes
```
$ git diff
```
Show unstaged changes between working directory and index.

Show Staged Changes
```
$ git diff --staged
```
Show changes between index and last commit (what will be committed).

View Commit Content
```
$ git show
```
Show metadata and content changes of a specific commit.

Staging & Commit — Add, commit, and manage changes

Stage a File
```
$ git add
```
Add a specific file to the staging area.

Stage All Changes
```
$ git add .
```
Stage all new and modified files in the current directory.

Stage All Including Deleted
```
$ git add -A
```
Stage all changes including deletions across the entire repo.

Stage Interactively
```
$ git add -p
```
Review and stage changes hunk by hunk (patch mode).

Commit Changes
```
$ git commit -m "message"
```
Commit staged changes with a descriptive message.

Commit with Details
```
$ git commit -m "title" -m "description"
```
Commit with a title and a detailed body message.

Amend Last Commit
```
$ git commit --amend
```
Modify the most recent commit (message or changes).

Amend Without Changing Message
```
$ git commit --amend --no-edit
```
Add staged changes to the last commit without editing the message.

Unstage a File
```
$ git restore --staged
```
Remove a file from the staging area (keep changes in working dir).

Discard Changes in File
```
$ git restore
```
Discard local changes in a file and restore to last committed state.

Branching — Create, switch, merge, rebase

List Branches
```
$ git branch
```
List all local branches. Current branch is marked with *.

Create Branch
```
$ git branch
```
Create a new branch from the current HEAD.

Create & Switch
```
$ git checkout -b
```
Create a new branch and switch to it immediately.

Switch Branch
```
$ git checkout
```
Switch to an existing branch.

Switch (New Syntax)

```
$ git switch
```

Modern alternative to checkout for switching branches.

Force Delete Branch

```
$ git branch -D
```

Force delete a branch even if not merged.

Merge Without Commit

```
$ git merge --no-commit --no-ff
```

Merge changes but stop before committing for review.

Rename Branch

```
$ git branch -m
```

Rename a local branch.

Delete Branch

```
$ git branch -d
```

Delete a branch (only if fully merged).

Merge Branch

```
$ git merge
```

Merge the specified branch into the current branch.

Rebase onto Branch

```
$ git rebase
```

Replay current branch commits on top of another branch.

Remote — Push, pull, fetch, manage remotes

List Remotes

```
$ git remote -v
```

List all configured remote repositories with URLs.

Change Remote URL

```
$ git remote set-url origin
```

Update the URL of an existing remote.

Pull Changes

```
$ git pull origin
```

Fetch and merge remote changes into current branch.

Push & Set Upstream

```
$ git push -u origin
```

Push and set the remote branch as the default tracking branch.

Delete Remote Branch

```
$ git push origin --delete
```

Delete a branch from the remote repository.

Add Remote

```
$ git remote add origin
```

Add a new remote repository named "origin".

Fetch Updates

```
$ git fetch origin
```

Download objects and refs from remote without merging.

Push Changes

```
$ git push origin
```

Upload local commits to the remote branch.

Force Push (Safe)

```
$ git push --force-with-lease
```

Safely force push — only if remote hasn\

View Remote Branches

```
$ git branch -r
```

List all remote-tracking branches.

Undo & Stash — Reset, revert, and stash

Undo Last Commit (Keep Changes)

```
$ git reset --soft HEAD~1
```

Undo the last commit but keep all changes staged.

Hard Reset to Commit

```
$ git reset --hard
```

Reset to a specific commit, discarding all changes after it.

Revert Multiple Commits

```
$ git revert ^..
```

Revert a range of commits in reverse order.

Undo Last Commit (Unstage)

```
$ git reset --mixed HEAD~1
```

Undo the last commit and unstage changes (default behavior).

Revert a Commit

```
$ git revert
```

Create a new commit that undoes the changes of a specific commit.

Stash Changes

```
$ git stash
```

Temporarily save uncommitted changes and clean the working directory.

Stash with Message

```
$ git stash push -m "description"
```

Save changes with a descriptive label.

Apply Stash

```
$ git stash pop
```

Restore the most recent stash and remove it from the list.

List Stashes

```
$ git stash list
```

Show all saved stashes.

Drop Stash

```
$ git stash drop stash@{n}
```

Delete a specific stash.

Advanced — Cherry-pick, bisect, reflog

Cherry-Pick Commit

```
$ git cherry-pick
```

Apply the changes from a specific commit to the current branch.

Interactive Rebase

```
$ git rebase -i HEAD~n
```

Edit, squash, reorder, or drop the last n commits interactively.

Find Commit by Content

```
$ git log -S "search text" --oneline
```

Search for commits that added or removed specific text.

Blame a File

```
$ git blame
```

Show who last modified each line of a file.

Tag a Release

```
$ git tag -a v1.0.0 -m "Release v1.0.0"
```

Create an annotated tag for a release.

Push Tags

```
$ git push origin --tags
```

Push all local tags to the remote repository.

Create Worktree

```
$ git worktree add ../fix-bug
```

Create a linked working directory for parallel work.

Show Reflog

```
$ git reflog
```

Show a log of all HEAD movements — recover lost commits.

Clean Untracked Files

```
$ git clean -fd
```

Remove untracked files and directories (dry-run with -n first!).

Bisect for Bugs

```
$ git bisect start
$ git bisect bad
$ git bisect good
```

Binary search through commit history to find the bug-introducing commit.

Reword Commit (Git 2.54+)

```
$ git history reword
```

Rewrite a commit message in place without touching working tree or index. Simpler than rebase -i.

Split Commit (Git 2.54+)

```
$ git history split
```

Interactively split a commit into two by selecting which hunks go to each.

Config — User, editor, credentials

Set User Name

```
$ git config --global user.name "Your Name"
```

Set the global author name for all commits.

Set User Email

```
$ git config --global user.email "email@example.com"
```

Set the global author email for all commits.

Set Default Branch

```
$ git config --global init.defaultBranch main
```

Change the default branch name from "master" to "main".

Set Default Editor

```
$ git config --global core.editor "vim"
```

Set the default text editor for commit messages.

Enable Autocorrect

```
$ git config --global help.autocorrect 1
```

Auto-run commands when there\

View All Config

```
$ git config --list
```

Display all current Git configuration settings.

Credential Helper (Cache)

```
$ git config --global credential.helper "cache --timeout=3600"
```

Cache credentials in memory for 1 hour.

Credential Helper (Store)

```
$ git config --global credential.helper store
```

Permanently store credentials on disk (less secure).

Workflow — Submodules, archives, cleanup

Create .gitignore

```
$ touch .gitignore
```

```
$ echo "node_modules/" >> .gitignore
```

Create and configure files/directories Git should ignore.

Add Submodule

```
$ git submodule add
```

Add another Git repository as a submodule.

Update Submodules

```
$ git submodule update --init --recursive
```

Initialize and update all submodules recursively.

Archive Repository

```
$ git archive --format=zip HEAD -o project.zip
```

Create a ZIP archive of the current HEAD without .git folder.

Contributor Stats

```
$ git shortlog -sn --all
```

Show commit count per contributor across all branches.

Compare Branches

```
$ git diff main..feature
```

Show all differences between two branches.

Find Merged Branches

```
$ git branch --merged
```

List branches that have been merged into the current branch.

Delete Merged Branches

```
$ git branch --merged | grep -v "\\*\\|main\\|master" | xargs git branch -d
```

Safely delete all branches merged into current (except main/master).

Show Ignored Files

```
$ git status --ignored
```

List files that are being ignored by .gitignore.

Sparse Checkout

```
$ git sparse-checkout init --cone
```

```
$ git sparse-checkout set
```

Checkout only specific directories from a large repository.

Support This Project

This cheatsheet is free and always will be.

If it saved you time, consider supporting.

Liberapay

`https://liberapay.com/inayatullahshinwari/donate`

Recurring donations, open-source friendly

Ko-fi

`https://ko-fi.com/inayatullahshinwari`

One-time tips, quick and easy

Crypto (USDT TRC20)

`TUFAAC9Zau3waUuHMnrPoaK92JbXp4YMag`

Direct wallet transfer, no fees

Thank you for keeping this project alive.

Built with ❤ by Inayatullah Shinwari

`https://inayatullahshinwari.github.io/git-cheatsheet`